

Regularization in Deep Learning | L1 & L2 Regularization | Weight Decay in ANN

➤ Introduction

Regularization is a **set of techniques** used to make a neural network **generalize better** — that means: perform well not just on training data, but also on **new/unseen data**.

When a model learns **too well** on training data (memorizes it), it **overfits**, and regularization helps prevent that.

➤ Improving Neural Network Performance

To improve neural network performance, we can:

1. Use **more data**
2. Tune **hyperparameters**
3. **Simplify** the model
4. Use **regularization** techniques (L1, L2, Dropout, Early Stopping, etc.)

Regularization ensures that the model learns **useful patterns**, not **noise**.

➤ What is Overfitting?

Overfitting occurs when the model:

- Has **very low error** on training data
- But **high error** on test data

This happens because the model learns small fluctuations or “noise” as if they were important patterns.

Example:

A student memorizes training examples instead of understanding the actual concept.

➤ Why Neural Networks Overfit?

Neural networks are powerful — they can learn **complex mappings** between inputs and outputs.

However:

- If the model is **too large** (too many neurons/layers)
 - Or training data is **too small**
 - The network memorizes the data perfectly — but fails to generalize.
-

➤ Ways to Solve Overfitting

1. **More training data**
-

2. **Simplify model architecture**
3. **Early stopping**
4. **Dropout**
5. **Data augmentation**
6. **Regularization (L1, L2, Weight Decay)** ← *focus today*

➤ What is Regularization?

Regularization **adds a penalty term** to the loss function to discourage the model from using **large weights**.

The idea:

- Large weights make the model complex and sensitive to input noise.
- Smaller weights make it smoother and more general.

➤ Modified Loss Function

$$L = L_0 + \lambda R(w)$$

Where:

- L_0 : Original loss (e.g., cross-entropy)
- $R(w)$: Regularization term (depends on L1 or L2)
- λ : Regularization strength (hyperparameter)

➤ Intuition Behind Regularization

- Regularization keeps the **weights small and balanced**.
- Prevents the model from assigning **too much importance** to specific features.
- Leads to smoother decision boundaries.

Think of it as **keeping the model “humble”** not letting it overreact to every detail in the data.

Types of Regularization

1. L1 Regularization (Lasso Regularization)

$$R(w) = \sum |w_i|$$

- Adds the **absolute value** of weights to the loss.
- Encourages **sparsity** (many weights become 0).

- Helps with **feature selection**.

Some neurons/connections become inactive → simpler model.

Example Code:

```
from tensorflow.keras.regularizers import l1

Dense(64, activation='relu', kernel_regularizer=l1(0.01))
```

2. L2 Regularization (Ridge Regularization / Weight Decay)

$$R(w) = \sum w_i^2$$

- Adds the **square of the weights** to the loss.
- Penalizes large weights more heavily.
- Keeps all weights small but **non-zero**.

Makes the model more stable and general.

Example Code:

```
from tensorflow.keras.regularizers import l2

Dense(64, activation='relu', kernel_regularizer=l2(0.001))
```

➤ Difference Between L1 and L2

Feature	L1 Regularization	L2 Regularization
Formula	sum(w)	$\sum w^2$
Effect	Makes some weights 0 (sparse)	Keeps all weights small
Use Case	Feature selection	Smooth weight decay
Another Name	Lasso	Ridge / Weight Decay

3. Weight Decay

Weight Decay is another name for **L2 Regularization**, especially when used in optimization algorithms (like SGD). It means weights **decay** slightly in every iteration:

$$w := w - \eta \frac{\partial L}{\partial w} - \eta \lambda w$$

➤ Visual Intuition

Without Regularization:

The model fits training data *too perfectly* — wiggly and unstable boundary.

With Regularization:

The model learns a smoother, simpler curve — better generalization.

➤ **Outro / Key Takeaways**

Concept	Explanation
Goal	Prevent overfitting and improve generalization
How	Add penalty to large weights
Main Types	L1 (sparse), L2 (small weights), Weight Decay
Tunable Parameter	λ (lambda) controls regularization strength
Effect	Simpler, smoother model with balanced weights

➤ **In Simple Words**

Regularization is like telling your model:

“Don’t rely too much on any single feature keep things simple and balanced.”
